



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

Lập trình Hướng Đối tượng Object Oriented Programming

\$7 Ghi đề; Lớp trừu tượng; Giao diện





Nội dung

- Ghi đè - Override
- Lớp trừu tượng - abstract class
- Giao diện - interface





ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

1. Ghi đè - Override

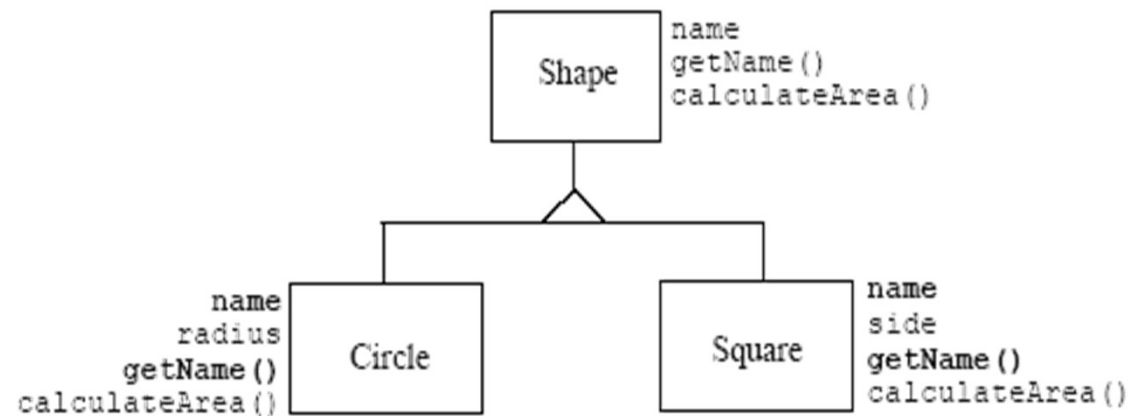


Mối quan hệ kế thừa

- Lớp con
 - Là một loại /dạng(is-a-kind-of) của lớp cha
 - Kế thừa các thành phần dữ liệu và các hành vi của lớp cha
 - Có thể thay đổi phù hợp với mục đích sử dụng mới
 - Extension: Thêm các thuộc tính/hành vi mới
 - Redefinition (Method Overriding): Chỉnh sửa lại các hành vi kế thừa từ lớp cha

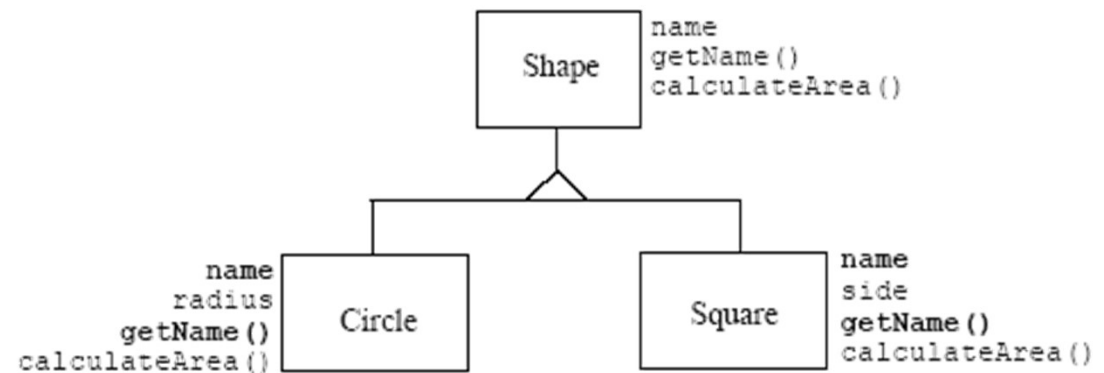
Định nghĩa lại / ghi đè

- Phương thức ghi đè sẽ thay thế hoặc làm rõ hơn cho phương thức cùng tên trong lớp cha
- Đối tượng của lớp con sẽ hoạt động với Phương thức mới phù hợp với nó



Định nghĩa lại / ghi đè

- Cú pháp: Phương thức ở lớp con hoàn toàn giống về chữ ký với phương thức ở lớp cha
 - Trùng tên & danh sách tham số
 - Mục đích: thể hiện cùng bản chất công việc



Ví dụ (C#)

- lớp cha

```
public class Employee
{
    public string Name { get; }
    protected decimal _basepay;
    public Employee(string name, decimal basepay)
    {
        Name = name;
        _basepay = basepay;
    }
    // Virtual => có thể override
    public virtual decimal CalculatePay()
    {
        return _basepay;
    }
}
```

Ví dụ (C#)

■ Lớp con

```
public class SalesEmployee : Employee
{
    private decimal _salesbonus; //Trường thu nhập đặc thù của sale
    // Constructor gọi đến base class và khởi tạo salesbonus
    public SalesEmployee(string name, decimal basepay, decimal salesbonus)
        : base(name, basepay)
    {
        _salesbonus = salesbonus;
    }
    // Override CalculatePay method: to take bonus into account.
    public override decimal CalculatePay()
    {
        return _basepay + _salesbonus;
    }
}
```

Ví dụ (C#)

■ thực thi

```
// Create some new employees.  
var employee1 = new SalesEmployee("Thanh", 1000, 500);  
var employee2 = new Employee("Mai", 1200);  
  
Console.WriteLine($"Employee1 {employee1.Name} earned: {employee1.CalculatePay()}");  
Console.WriteLine($"Employee2 {employee2.Name} earned: {employee2.CalculatePay()}");  
}  
}  
/*  
Output:  
    Employee1 Thanh earned: 1500  
    Employee2 Mai earned: 1200  
*/
```

Ví dụ (C#)

```
// Base class
public class Shape
{
    // Virtual method to be overridden in derived classes
    public virtual double CalculateArea()
    {
        return 0;
    }
}
```

```
public class Circle : Shape
{
    public double Radius { get; set; }
    public Circle(double radius)
    {
        Radius = radius;
    }
    // Overriding CalculateArea method
    public override double CalculateArea()
    {
        return Math.PI * Math.Pow(Radius, 2);
    }
}
```

VD

- Viết lớp Triangle => bài tập: override method

```
// Base class
public class Shape
{
    // Virtual method to be overridden in derived classes
    public virtual double CalculateArea()
    {
        return 0;
    }
}
```

Ví dụ (Java)

- Lớp cơ sở

```
class Shape {  
    protected String name;  
    Shape(String n) { name = n; }  
    public String getName() { return name; }  
    public float calculateArea() { return 0.0f; }  
}
```

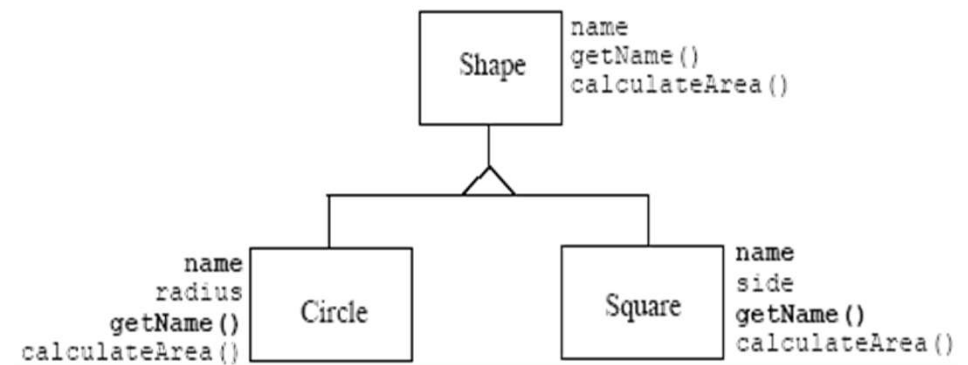
- Lớp circle

```
class Circle extends Shape {  
    private int radius;  
    Circle(String n, int r){  
        super(n);  
        radius = r;  
    }  
    public float calculateArea() {  
        float area = (float)(3.14 * radius * radius); return area;  
    }  
}
```

Ví dụ (Java)

- Lớp square

```
class Square extends Shape
{
    private int side;
    Square(String n, int s) {
        super(n);
        side = s;
    }
    public float calculateArea() {
        float area = (float) side * side;
        return area;
    }
}
```



Gọi Phương thức lớp cha

- Lớp con thực hiện override & có thể tái sử dụng các đoạn mã của lớp cha
 - Gọi constructor
 - Gọi các method của lớp cha

```
public class SalesEmployee : Employee
{
    private decimal _salesbonus;
    public SalesEmployee(string name, decimal basepay, decimal salesbonus)
        : base(name, basepay) {
        _salesbonus = salesbonus;
    }
    public override decimal CalculatePay(){
        return base.CalculatePay() + _salesbonus;
    } ...
}
```

Gọi Phương thức lớp cha

- Java: Sử dụng từ khóa super

- gọi constructor lớp cha

`super`(danh sách tham số);

- gọi method lớp cha

`super.method_name`(danh sách tham số);

```
package abc;
public class Person {
    protected String name;
    protected int age;
    public String getDetail() {
        String s = name + "," + age;
        return s;
    }
}
```

```
import abc.Person;
public class Employee extends Person {
    double salary;
    public String getDetail() {
        String s = super.getDetail() + "," + salary;
        return s;
    }
}
```

Quy định trong ghi đè

- Phương thức ghi đè trong lớp con phải
 - Có danh sách tham số giống phương thức kế thừa trong lớp cha.
 - Có cùng kiểu trả về với phương thức kế thừa trong lớp cha
- Java:
 - Các chỉ định truy cập không giới hạn chặt hơn phương thức trong lớp cha.
Ví dụ, nếu ghi đè một phương thức protected, thì phương thức mới có thể là protected hoặc public, mà không được là private.
- C#:
 - Chỉ định truy cập phải giống nhau

Quy định trong ghi đề (tiếp)

- Không được phép ghi đề:
 - Các phương thức static trong lớp cha
 - Các phương thức private trong lớp cha
 - Java: Các phương thức hằng (final) trong lớp cha

Hạn chế ghi đè: Java

- Đôi lúc cần hạn chế việc định nghĩa lại vì các lý do sau:
 - Tính đúng đắn: Định nghĩa lại một phương thức trong lớp dẫn xuất có thể làm sai lệch ý nghĩa của nó
 - Tính hiệu quả: Cơ chế kết nối động không hiệu quả về mặt thời gian bằng kết nối tĩnh. Nếu biết trước sẽ không định nghĩa lại phương thức của lớp cơ sở thì nên dùng từ khóa final đi với phương thức

```
public final String baseName () {  
    return "Person";  
}
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

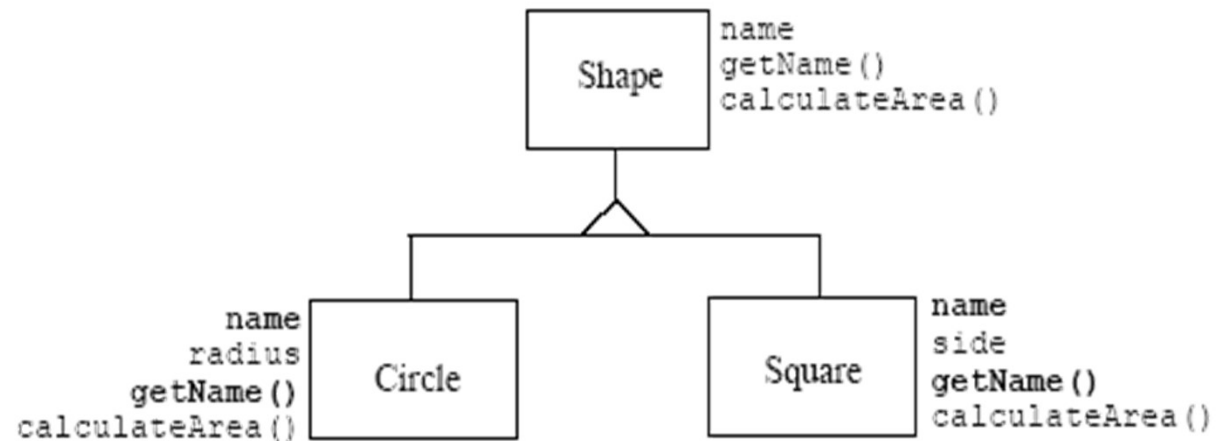
KHOA TOÁN – TIN

2. Lớp trừu tượng - Abstract class



Lớp trừu tượng

- Ví dụ xét lớp Shape
 - Lớp "không rõ ràng"
 - Nói chung không có đối tượng trực tiếp
- => thiết kế lớp trừu tượng



Lớp trừu tượng

■ Mục đích

- Định nghĩa bản mẫu/bản thiết kế các khái niệm chung => các lớp kế thừa sẽ triển khai
- Dùng lớp trừu tượng định nghĩa tập hợp chung các hành vi, thuộc tính - các lớp dẫn xuất phải có
- Thể hiện tính đa hình của OOP

Lớp trừu tượng

- Đặc điểm
 - Không thể tạo đối tượng từ abstract class (new Class() => error)
 - Thường cũng định nghĩa các abstract method
- Phương thức trừu tượng - abstract method
 - Chỉ có chữ ký/định nghĩa mà không có cài đặt => các lớp dẫn xuất cài đặt/định nghĩa/override

Lớp trừu tượng

- Nếu một lớp có một hay nhiều phương thức trừu tượng thì nó phải là lớp trừu tượng
 - Lớp con khi kế thừa phải cài đặt cụ thể cho các phương thức trừu tượng của lớp cha
- => Phương thức trừu tượng không thể khai báo là static.

Kết hợp hợp lệ
abstract public
abstract protected

Kết hợp KHÔNG hợp lệ
abstract private
abstract static
abstract final (Java)

Cú pháp

- Thường là
 - Từ khóa `abstract`
 - Lớp trừu tượng

```
public abstract class Shape
{
    // ....
}
```

Shape a = new Shape (); // Error: Cannot create an instance of the abstract class

- Phương thức trừu tượng

```
public abstract class Shape
{
    //Abstract method
    public abstract double CalculateArea();
}
```

Ví dụ (C#)

```
public class Circle : Shape
```

```
{
```

```
    public double Radius { get; set; }
```

```
    public Circle(double radius){
```

```
        Radius = radius;
```

```
    }
```

```
    // Implement the CalculateArea method
```

```
    public override double CalculateArea() {
```

```
        return Math.PI * Radius * Radius;
```

```
    }
```

```
}
```

```
public abstract class Shape
```

```
{
```

```
    public string name { get; set; }
```

```
    public String getName() { return name; }
```

```
    //Abstract method => implemented in derived
```

```
    public abstract double CalculateArea();
```

```
}
```

```
Circle circle = new Circle(5.0);
```

```
circle.name = "Circle 1";
```

```
circle.getName();
```

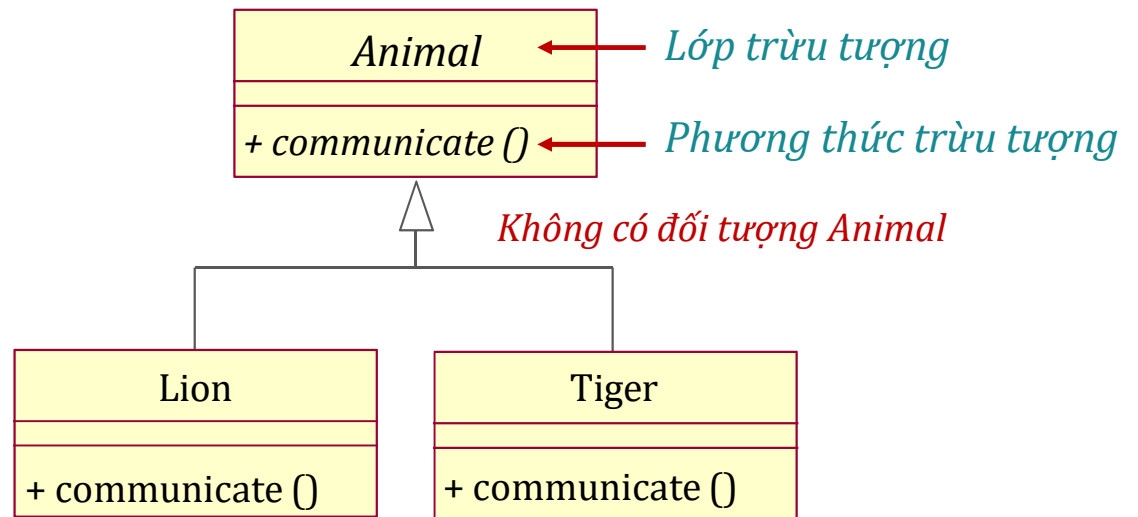
```
Console.WriteLine($"Dien tich: {circle.CalculateArea()}");
```

C#: abstract >< virtual

Abstract method	Virtual method
Lớp con bắt buộc phải override abstract method ở lớp cha.	Lớp con không bắt buộc phải override virtual method ở lớp cha. Nếu định nghĩa của virtual method ở lớp cha đã phù hợp với lớp con rồi thì có thể dùng luôn. Còn chưa phù hợp thì lớp con có quyền override virtual method ở lớp cha.
Tuyệt đối chỉ được khai báo là prototype ở lớp cha không có thân hàm, tức là chưa được định nghĩa.	Cần phải được định nghĩa ở lớp cha.
Chỉ được sử dụng trong abstract class.	Có thể sử dụng được trong cả abstract class và normal class.

Biểu diễn UML

- Chữ *ngghiêng* italic



Tất cả các đối tượng thuộc lớp Lion hoặc Tiger



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

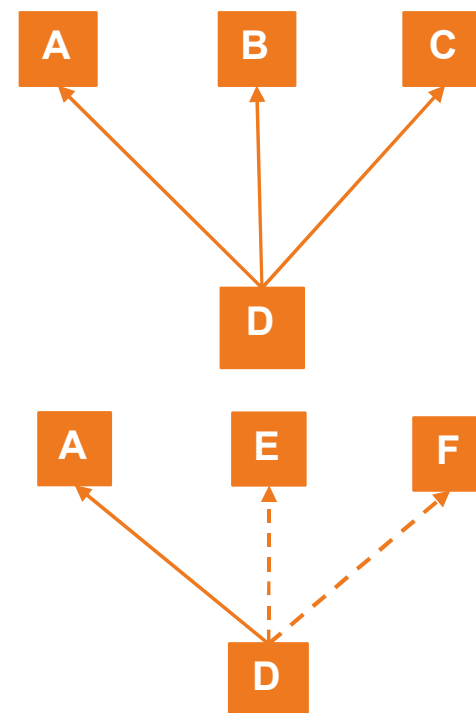
KHOA TOÁN – TIN

3. Giao diện - Interface



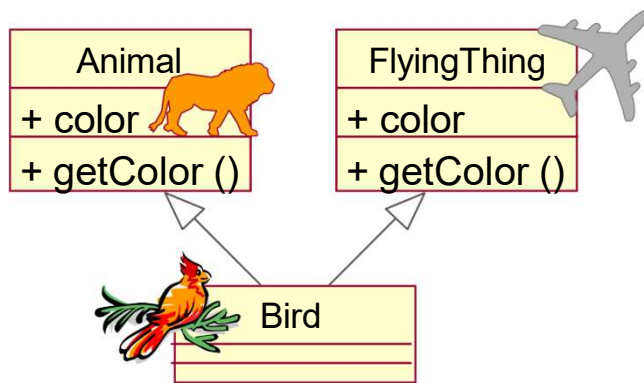
Đa kế thừa và đơn kế thừa

- Đa kế thừa
 - Một lớp con kế thừa nhiều lớp cha
 - Đơn kế thừa
 - Một lớp con kế thừa từ chỉ 1 lớp cha
 - Hỗ trợ của ngôn ngữ:
 - C++: hỗ trợ đa kế thừa
 - Java, C#: chỉ hỗ trợ đơn kế thừa
- => giải quyết phần nào qua Interface

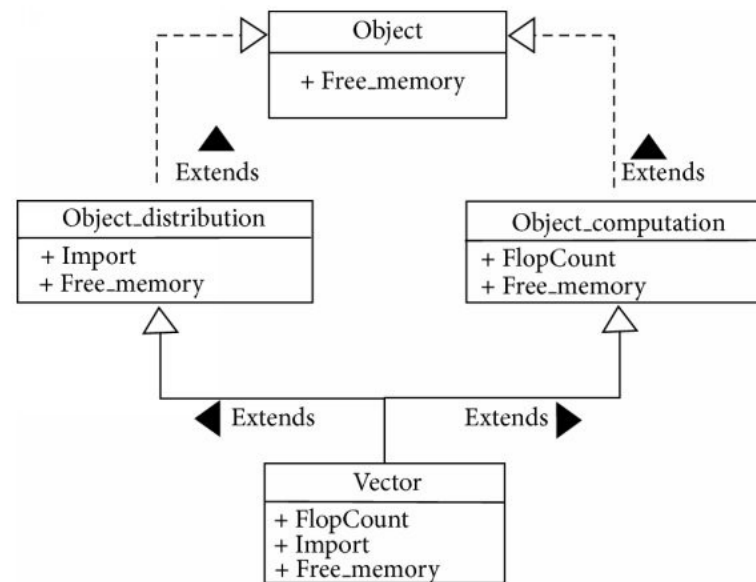


Vấn đề gặp phải trong đa kế thừa

Xung đột tên - Name collision



Vấn đề "Diamond shape"



Giao diện

- Giao diện-Interface là "hợp đồng" xác định các hành vi, đặc tính mà lớp đối tượng cần phải tuân theo
- Giao diện - Interface
 - Chứa các mô tả (chữ ký) của method (abstract method) và các const
 - Hỗ trợ giải quyết vấn đề đa kế thừa, tránh nhập nhằng

Giao diện

- Đặc tả giao diện - khá giống abstract class

- từ khóa interface (Java, C# ...)

- thành phần:

- các đặc tả method

- các thuộc tính

- Cú pháp:

```
interface <interface_name>  
interface <child_interface> : <parent_interface>
```

```
interface <interface_name>  
<child_interface> extends <parent_interface>
```

- VD

```
public interface IShape  
{  
    double CalculateArea();  
}
```

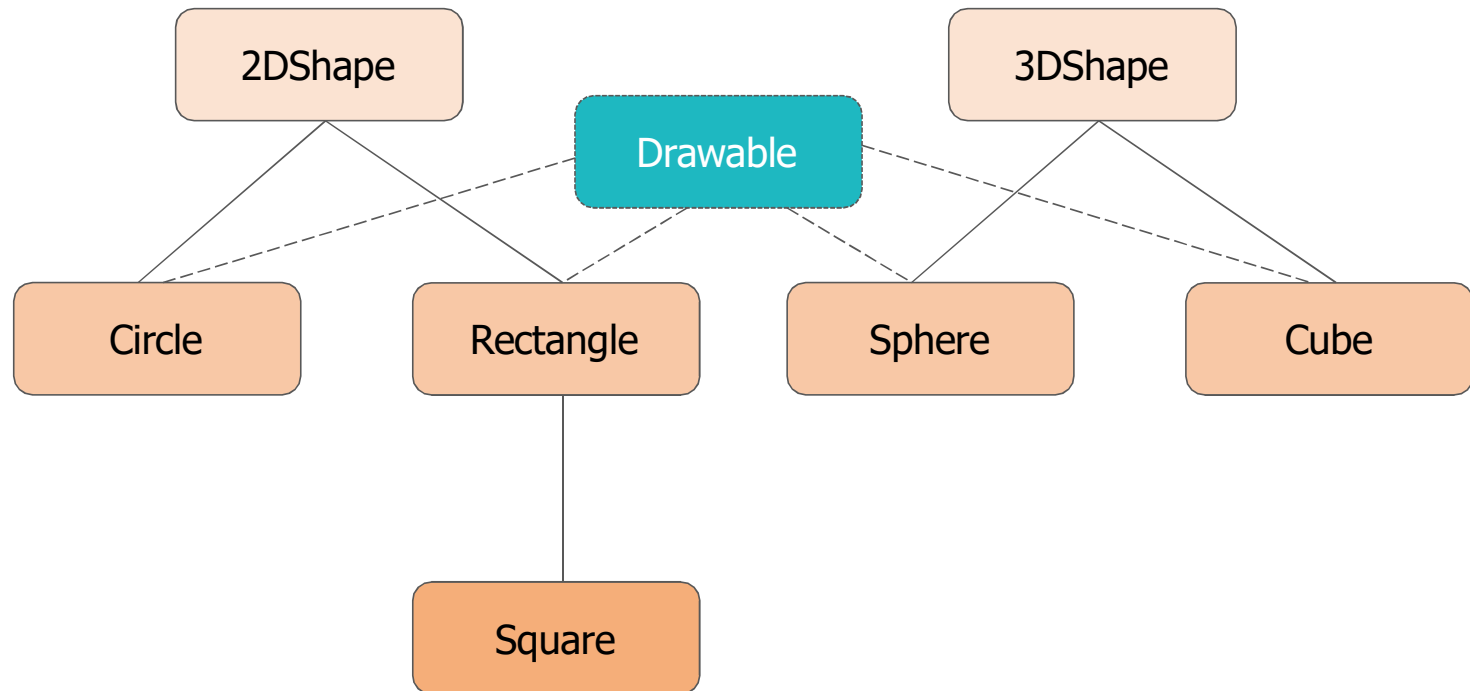
Lớp thực thi giao diện

- Lớp thực thi giao diện - triển khai cài đặt (how)
- Lớp thực thi
 - Lớp cài đặt tất cả các method được định nghĩa trong interface
 - Hoặc là lớp trừu tượng abstract
- Cú pháp: `<ImpClass> implements <List of interfaces>`
`public class <ImpClass> : <List of interfaces>`

- VD

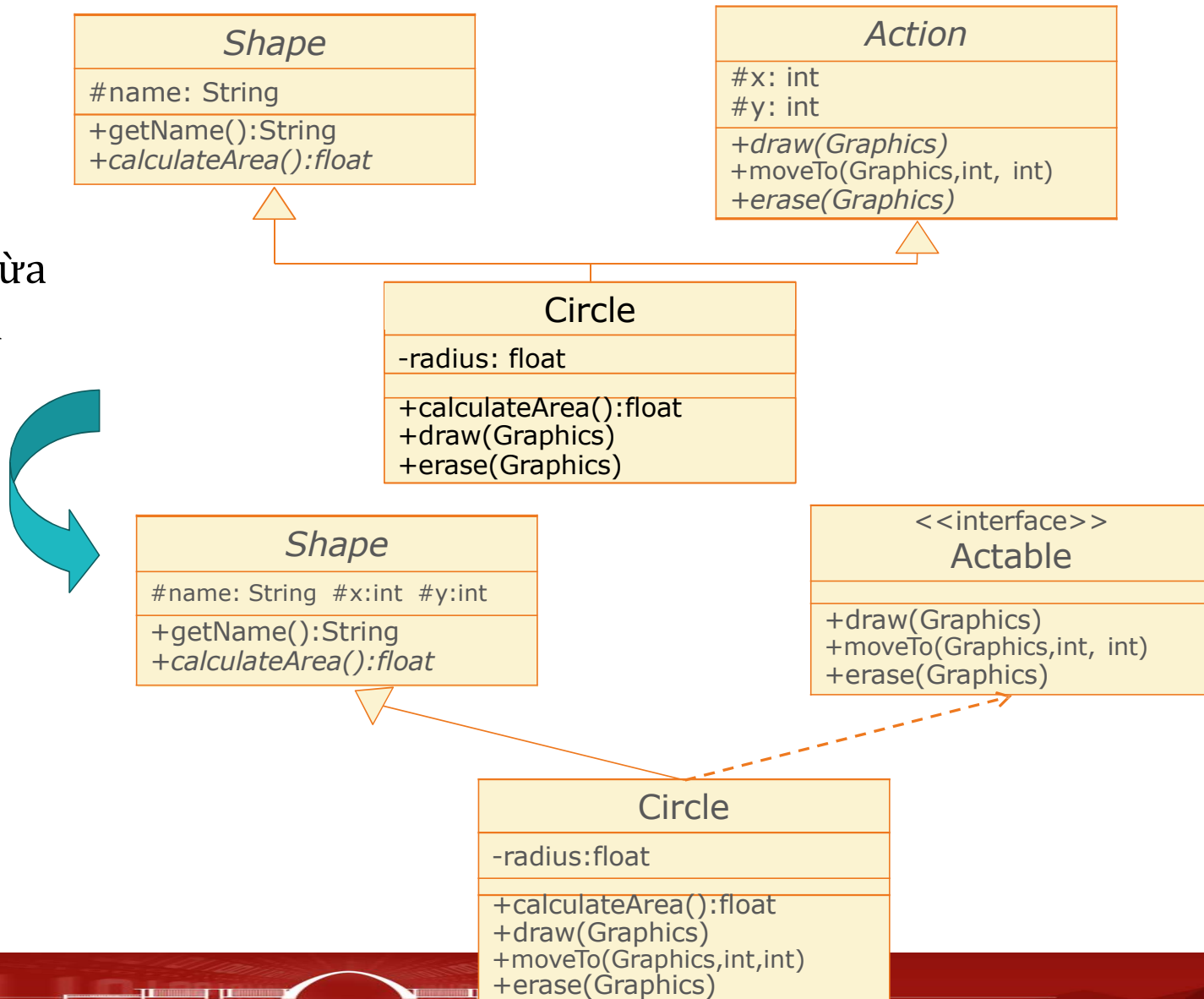
```
public class Rectangle : IShape {  
    public double Width { get; set; }  
    public double Height { get; set; }  
    // Implement the CalculateArea method  
    public double CalculateArea(){  
        return Width * Height;  
    }  
}
```

Interface



Giao diện & cài đặt

- Lớp có thể vừa kế thừa từ lớp cha và vừa cài đặt interface
- Lớp có thể cài đặt nhiều interface



Ví dụ (Java)

```
import java.awt.Graphics;
abstract class Shape {
    protected String name;
    protected int x, y;
    Shape(String n, int x, int y) {
        name = n; this.x = x; this.y = y;
    }
    public String getName() {
        return name;
    }
    public abstract float calculateArea();
}
interface Actable {
    public void draw(Graphics g);
    public void moveTo(Graphics g, int x1, int y1);
    public void erase(Graphics g);
}
```

Ví dụ (Java)

```
class Circle extends Shape implements Actable {
    private int radius;
    public Circle(String n, int x, int y, int r) {
        super(n, x, y); radius = r;
    }
    public float calculateArea() {
        float area = (float) (3.14 * radius * radius);
        return area;
    }
    public void draw(Graphics g) {
        System.out.println("Draw circle at ("
                           + x + ", " + y + ")");
        g.drawOval(x-radius, y-radius, 2*radius, 2*radius);
    }
    public void moveTo(Graphics g, int x1, int y1) {
        erase(g); x = x1; y = y1; draw(g);
    }
    public void erase(Graphics g) {
        System.out.println("Erase circle at ("
                           + x + ", " + y + ")");
        // paint the region with background color...
    }
}
```

Đặc điểm của Interface

- Interface có thể xem như dạng abstract class:
 - các method, prop là public
 - các method là abstract, prop là static
- Không hiện thực hóa-instantiate: không có đối tượng thể hiện
- Một class có thể thực hiện/cài đặt nhiều giao diện
- Giao diện có thể kế thừa

Đặc điểm của Interface

- Interface không cài đặt Phương thức nào
- Interface đưa ra thiết kế, khuôn mẫu, đặc tả/contract;
 - Các nhóm phát triển phần mềm thống nhất sản phẩm của họ tương tác với nhau như thế nào, mà không đòi hỏi bất cứ một thông tin nào về cách thức cài đặt chi tiết
- Cho phép tách biệt đặc tả (what) và cài đặt (how)



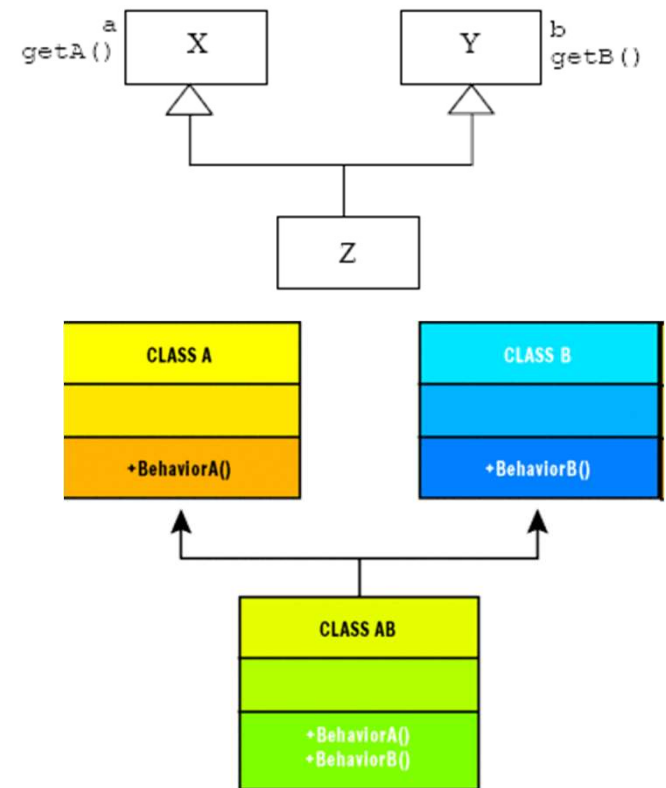
Lợi ích của Interface

- Giải quyết đa kế thừa
- Đưa ra quy tắc, hành vi chung



Nhược điểm

- Thay đổi Interface => thay đổi tất cả các lớp cài đặt
- Interface không có tái sử dụng mã lệnh



Interface => đa thừa kế

```
// Interface for account operations
public interface IAccount
{
    void Deposit(double amount);
    void Withdraw(double amount);
}

// Interface for reporting
public interface IReport
{
    string GenerateReport();
}
```

Interface => đa thừa kế

```
// Class implementing both interfaces
public class BankAccount : IAccount, IReport
{
    public int AccountID { get; set; }
    public string AccountHolder { get; set; }
    public double Balance { get; private set; }
    // Implement the Deposit method
    public void Deposit(double amount){
        Balance += amount;
    }
    // Implement the Withdraw method
    public void Withdraw(double amount){
        Balance -= amount;
    }
    // Implement the GenerateReport method
    public string GenerateReport(){
        return $"Account ID: {AccountID}\nBalance: {Balance:C}";
    }
}
```

Interface => đa thừa kế

```
// Usage example
public class Program
{
    public static void Main()
    {
        BankAccount account = new BankAccount(123456, "Thang", 10000000);
        account.Deposit(500000.0);
        account.Withdraw(200000.0);
        Console.WriteLine(account.GenerateReport());
    }
}
```

Ví dụ Java 8 Interface – default methods

- Java 8 giới thiệu một khái niệm mới là *default* method dành cho interface, cho phép chúng ta thêm vào các chức năng cho interface mà không làm phá vỡ các lớp implement từ interface này
— Ngầm định là *public*

```
public interface Shape {  
  
    void draw();  
  
    default void setColor(String color) {  
        System.out.println("Draw shape with color " + color);  
    }  
}
```

Ví dụ Đa thừa kế

```
interface Interface1 {  
    default void doSomething() {  
        System.out.println("doSomething1");  
    }  
}  
  
interface Interface2 {  
    default void doSomething() {  
        System.out.println("doSomething2");  
    }  
}  
  
public class MultiInheritance implements  
Interface1, Interface2 {  
    @Override  
    public void doSomething() {  
        Interface1.super.doSomething();  
    }  
}
```

Ví dụ Đa thừa kế

```
interface Interface3 {  
    default void doSomething() {  
        System.out.println("Execute in  
Interface3");  
    }  
}  
  
class Parent {  
    public void doSomething() {  
        System.out.println("Execute in Parent");  
    }  
}  
  
public class MultiInheritance2 extends Parent  
implements Interface3 {  
    public static void main(String[] args) {  
        MultiInheritance2 m = new  
MultiInheritance2();  
        m.doSomething(); // Execute in Parent  
    }  
}
```

Ví dụ Java 8 interface – Static methods

```
interface Vehicle {  
    default void print() {  
        if (isValid())  
            System.out.println("Vehicle printed");  
    }  
    static boolean isValid() {  
        System.out.println("Vehicle is valid");  
        return true;  
    }  
    void showLog();  
}  
  
public class Car implements Vehicle {  
    @Override  
    public void showLog() {  
        print();  
        Vehicle.isValid();  
    }  
}
```

Lớp trừu tượng >< Giao diện

	Interface	Abstract class
Multiple inheritance	Một class có thể hiện thực nhiều interface.(tạm coi là thừa kế)	Không hỗ trợ đa thừa kế
Default implementation	Không thể định nghĩa code xử lý, chỉ có thể khai báo.	Có thể định nghĩa thân phương thức, property.
Access Modifiers	Mọi phương thức, property đều mặc định là public.	Có thể xác định modifier.
Adding functionality	Mọi phương thức, property của interface cần được hiện thực trong class.	Không cần thiết.
Fields and Constants	Không	Có

Lớp trừu tượng & Giao diện

- Khi nào nên cho một lớp là lớp độc lập, lớp con, lớp trừu tượng, hay nên biến nó thành interface?
 - Một lớp nên là lớp độc lập, nghĩa là nó không thừa kế lớp nào nếu nó không thỏa mãn quan hệ IS-A đối với bất cứ loại nào khác
 - Một lớp nên là lớp con nếu cần cho nó làm một phiên bản chuyên biệt hơn của một lớp khác và cần ghi đè hành vi có sẵn hoặc bổ sung hành vi mới

Lớp trừu tượng & Giao diện

- Khi nào nên cho một lớp là lớp độc lập, lớp con, lớp trừu tượng, hay nên biến nó thành interface?
 - Một lớp nên là lớp cha nếu muốn định nghĩa một khuôn mẫu cho một nhóm các lớp con, và có mã cài đặt mà tất cả các lớp con kia có thể sử dụng
 - Cho lớp đó làm lớp trừu tượng nếu muốn đảm bảo rằng không ai được tạo đối tượng thuộc lớp đó
 - Dùng một interface nếu muốn định nghĩa vai trò/hành vi mà các lớp khác phải tuân thủ, bất kể các lớp đó thuộc cây thừa kế nào

Tổng kết

- Override:
 - định nghĩa lại hành vi ở lớp con
- Abstract class
 - Không khởi tạo được đối tượng, được tạo ra làm lớp cơ sở để lớp con định nghĩa rõ hơn
- Interface
 - Định nghĩa/đặc tả thiết kế/contract mà lớp thực thi phải tuân thủ (cài đặt)
 - Giải quyết đa kế thừa

- Bài tập Interface: transportation vehicles

- + phương tiện đường bộ (RoadVehicle) have ability to start, turnleft, turnright, stop
 - + WaterVehicle have ability sail,
 - + Air vehicle: fly, takeoff, landing
 - + cài đặt các lớp car, ship, hybrid (phương tiện lưỡng cư thủy bộ), taxi bay (lưỡng cư bộ, hàng không)
- 